IF4031: ARSITEKTUR APLIKASI TERDISTRIBUSI

LAPORAN TUGAS BESAR MILESTONE 1

Pengembangan Platform Koordinasi Kebencanaan BNPB

Sistem Terdistribusi Dasar Multi-Sumber (BMKG & PVMBG)

Nama Kelompok: AhhAldoyTolong

Dipersiapkan oleh:

13523061 Darrel Adinarya Sunanda
13524110 Jennifer Khang
18223113 Aldoy Fauzan Avanza

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Tabel Kontribusi iv

1 Pendahuluan **1**
1.1 Deskripsi Umum Sistem 1
1.2 Tujuan Sistem 2
1.3 Asumsi Perancangan dan Implementasi 3

2 Arsitektur dan Teknologi Sistem **4**
2.1 Arsitektur Sistem 4
2.2 Tanggung Jawab Setiap Service 4
2.3 Protokol Komunikasi Antarservice 6
2.4 Kepemilikan Penyimpanan dan Batas Container 6
2.5 Alur Permintaan, Data, dan Event 6
2.6 Teknologi Terpilih dan Alasan Pemilihan 7

3 Solusi dan Pembahasan Permasalahan **9**
3.1 Problem 1: Arsitektur dan Interoperabilitas Data 9
3.1.1 Rancangan Solusi dan Mekanisme 9
3.1.2 Alasan Keputusan dan Trade-Off 10
3.1.3 Batasan yang Disadari 10
3.1.4 Bukti Penyelesaian 11
3.1.5 Jawaban Butir Spesifikasi (Di Laporan) 11
3.2 Problem 2: Concurrency, Availability, dan Graceful Degradation 12
3.2.1 Rancangan Solusi dan Mekanisme 12
3.2.2 Alasan Keputusan dan Trade-Off 13
3.2.3 Batasan yang Disadari 13
3.2.4 Bukti Penyelesaian 13
3.2.5 Jawaban Butir Spesifikasi (Di Laporan) 14
3.3 Problem 3: Pengendalian Hak Akses dan Isolasi Kredensial . . . 15
3.3.1 Rancangan Solusi dan Mekanisme 15

3.3.2 Alasan Keputusan dan Trade-Off 16
3.3.3 Batasan yang Disadari 16
3.3.4 Bukti Penyelesaian 16
3.3.5 Jawaban Butir Spesifikasi (Di Laporan) 17
3.4 Problem 4: Arsitektur Microservice dan Penyimpanan Fleksibel 18
3.4.1 Rancangan Solusi dan Mekanisme 18
3.4.2 Alasan Keputusan dan Trade-Off 18
3.4.3 Batasan yang Disadari 18
3.4.4 Bukti Penyelesaian 19
3.4.5 Jawaban Butir Spesifikasi (Di Laporan) 20
3.5 Problem 5: Fan-out Event Distribution atau Publish–Subscribe 21
3.5.1 Rancangan Solusi dan Mekanisme 21
3.5.2 Alasan Keputusan dan Trade-Off 23
3.5.3 Batasan yang Disadari 24
3.5.4 Bukti Penyelesaian 24
3.5.5 Jawaban Butir “Di Laporan” 26

4 Petunjuk Menjalankan Sistem 28
4.1 Prasyarat dan Konfigurasi Lingkungan 28
4.2 Langkah Menjalankan Sistem 29
4.3 Langkah Verifikasi dan Demonstrasi Solusi (Problem 1–5) 29

Daftar Pustaka 31

Lampiran A Pernyataan Penggunaan Kecerdasan Buatan (AI) 32

Daftar Tabel

1 Tabel Kontribusi Anggota Kelompok iv

2.1 Rangkuman Pemilihan Teknologi Sistem Terdistribusi BNPB . . 7

3.1 Evaluasi Komparatif Alternatif Arsitektur Penyimpanan 19

4.1 Matriks Verifikasi dan Target Eksekusi Demonstrasi Permasalahan 30

Daftar Gambar

2.1 Diagram Arsitektur Sistem Terdistribusi BNPB Milestone 1 . . 5

Tabel Kontribusi

Berikut disajikan rincian pembagian tanggung jawab dan kontribusi masing-masing anggota kelompok dalam perancangan arsitektur, implementasi *service*, pengujian *proof-of-concept*, serta penyusunan laporan Milestone 1:

Tabel 1: Tabel Kontribusi Anggota Kelompok

NIM	**Nama**	**Kontribusi**
13523061	Darrel Adinarya Sunanda	Mengembangkan layanan agregator untuk mengambil data dari BMKG dan PVMBG. Menyeragamkan data ke dalam skema kanonis. Memperkaya data menggunakan katalog gunung api statis. Mengembangkan API internal agregator. Mengelola penyimpanan PostgreSQL dengan atribut JSONB. Menerapkan mekanisme Transactional Outbox. Menyusun analisis konkurensi (P2) dan evaluasi penyimpanan fleksibel (P4).
13524110	Jennifer Khang	Mengembangkan layanan autentikasi dan penerbitan token akses (Auth Service). Menerapkan pengelolaan cakupan hak akses (RBAC scope). Menerapkan masa berlaku token akses 60 detik dan rotasi refresh token. Mengembangkan API publik (Client API) dengan pembatasan data berdasarkan hak akses. Menerapkan pelacakan permintaan menggunakan correlation ID. Mengembangkan konsumen kejadian untuk dasbor dan notifikasi dengan kelompok konsumen independen. Menyusun analisis keamanan (P3) dan pub-sub (P5).
18223113	Aldoy Fauzan Avanza	Mengembangkan layanan simulasi BMKG untuk data gempa dan peringatan tsunami. Menerapkan autentikasi kunci API pada layanan simulasi BMKG. Mengembangkan layanan simulasi PVMBG untuk laporan aktivitas gunung api. Menerapkan autentikasi Bearer token pada layanan simulasi PVMBG. Menyediakan mekanisme perubahan skema dinamis saat layanan berjalan. Menyediakan mekanisme simulasi gangguan sumber data. Menyusun analisis evolusi skema (P1).

Bab 1

Pendahuluan

1.1 Deskripsi Umum Sistem

Platform Koordinasi Kebencanaan BNPB merupakan prototipe aplikasi terdistribusi yang dirancang untuk mengintegrasikan informasi kebencanaan dari beberapa sumber serta menyediakan akses data sesuai kebutuhan pengguna. Sistem mengolah data gempa bumi dan peringatan tsunami dari layanan simulasi BMKG serta laporan aktivitas gunung api dari layanan simulasi PVMBG. Data yang memiliki format berbeda tersebut dikumpulkan oleh layanan agregator, diseragamkan ke dalam skema kanonik, dan disimpan agar dapat diakses melalui antarmuka yang konsisten.

Fungsi utama sistem meliputi pengambilan data secara berkala, penyeragaman struktur data, pengayaan laporan aktivitas gunung api menggunakan katalog referensi statis, serta penyediaan API publik dengan pembatasan akses. Sistem membedakan pengguna menjadi tiga kelompok, yaitu:

1. **Kelompok Media:** Memperoleh akses terbatas hanya terhadap informasi ringkasan (*summary*). Akses terhadap atribut mentah atau koordinat presisi dibatasi secara ketat di sisi server.
2. **Petugas Lapangan (BASARNAS / Pemda):** Memperoleh cakupan akses penuh (*raw/all*) menggunakan token akses berumur pendek (default 60 detik) yang dilengkapi mekanisme rotasi *refresh token*.
3. **Pengguna Internal Ops BNPB:** Memperoleh hak akses tertinggi untuk membaca seluruh data kanonik dan atribut spesifik.

Selain menyediakan jalur akses berbasis permintaan melalui HTTP/REST, sistem menggunakan mekanisme *publish–subscribe* berbasis message broker Redpanda untuk mendistribusikan kejadian kebencanaan kepada *consumer* hi-

lir secara asinkron.

Untuk Milestone 1, lingkup mencakup pengembangan layanan simulasi sumber data (*mock-bmkg* dan *mock-pvmbg*), agregator, penyimpanan kanonis, autentikasi (*auth*), API publik (*client-api*), serta dua konsumen kejadian (*dashboard-consumer* dan *notifier-consumer*). Perancangan dan demonstrasi sistem berfokus pada lima permasalahan utama:

- **Problem 1 (P1):** Perubahan skema data dinamis saat sistem berjalan (*live schema evolution*).
- **Problem 2 (P2):** Penanganan gangguan sumber data dan permintaan konkuren (*concurrency, availability, graceful degradation*).
- **Problem 3 (P3):** Pengendalian hak akses dan isolasi domain kepercayaan (*cross-credential trust isolation & RBAC*).
- **Problem 4 (P4):** Isolasi kegagalan container dan penyimpanan kanonis fleksibel (*independent container rebuilds & flexible storage*).
- **Problem 5 (P5):** Distribusi kejadian *publish–subscribe* dan pemulihan konsumen (*event fan-out & downtime recovery*).

Setiap layanan dijalankan dalam container Docker mandiri dan berkomunikasi secara eksklusif melalui panggilan jaringan (HTTP/REST atau message broker).

1.2 Tujuan Sistem

Sistem dikembangkan dengan tujuan:

1. Memahami perancangan arsitektur aplikasi terdistribusi melalui penentuan batas tanggung jawab *service*, komunikasi jaringan antarlayanan, kepemilikan penyimpanan independen, dan pengoperasian container secara terisolasi.
2. Menganalisis keputusan arsitektur beserta alternatif dan *trade-off*-nya dalam menangani interoperabilitas data, evolusi skema, konkurensi, ketersediaan tinggi, autentikasi terdesentralisasi, dan distribusi *event*.
3. Memvalidasi rancangan arsitektur melalui implementasi *proof-of-concept* dan verifikasi empiris terhadap perubahan skema dinamis, latensi, kegagalan layanan hulu, pembatasan akses berbasis peran, serta pemulihan konsumen setelah *downtime*.

1.3 Asumsi Perancangan dan Implementasi

Dalam perancangan dan implementasi Milestone 1, kelompok menetapkan sejumlah asumsi batasan:

1. **Sifat Akses Klien Hilir:** Seluruh klien hilir (Media, Petugas Lapangan, Internal) pada Milestone 1 bersifat *read-only*. Jalur pelaporan balik atau pembaruan status dari lapangan ke BNPB tidak termasuk dalam cakupan Milestone 1.
2. **Otonomi Sumber Data Hulu:** Layanan BMKG dan PVMBG beroperasi sebagai entitas independen yang bersifat pasif. Keduanya tidak melakukan *push notification* ke BNPB, melainkan diakses secara berkala melalui *polling* dengan jendela *overlap* 10 detik guna mencegah kehilangan data.
3. **Infrastruktur Redpanda:** Pada Milestone 1, broker Redpanda dijalankan dalam konfigurasi *single-broker* ter-kontainerisasi dengan satu partisi per topik atau 3 partisi replikasi lokal, yang mencukupi untuk simulasi *fan-out* dan akumulasi *lag*.
4. **Penyimpanan State Sesi Klien:** Penyimpanan sesi *refresh token* pada layanan Auth serta status deduplikasi pada *consumer* diimplementasikan secara *in-memory* lokal proses, yang disadari akan diekspansi ke penyimpanan terdistribusi (*distributed cache*) pada Milestone 2 saat replikasi horizontal diterapkan.

Bab 2

Arsitektur dan Teknologi Sistem

2.1 Arsitektur Sistem

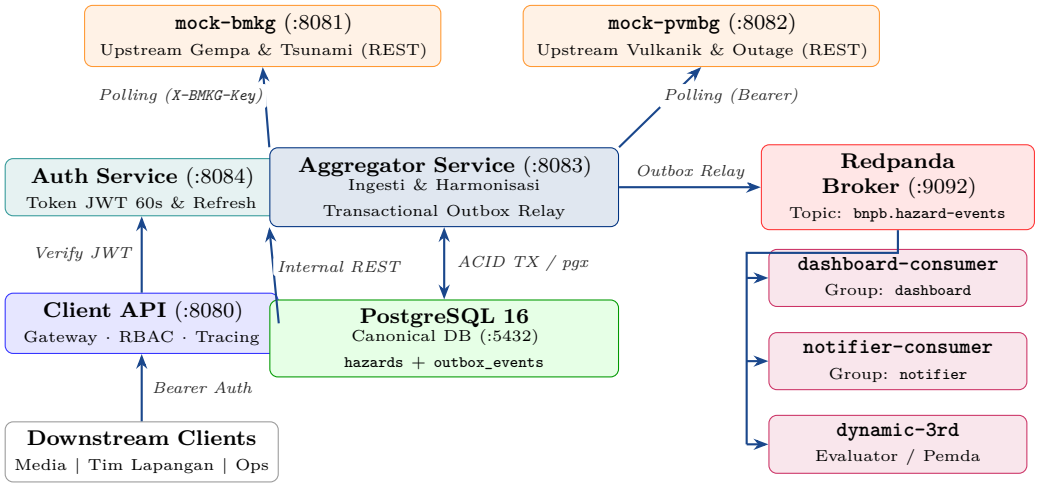
Sistem Koordinasi Kebencanaan BNPB dibangun menggunakan arsitektur *event-driven microservices* yang terisolasi dalam batas container mandiri. Arsitektur sistem terdiri atas tiga tingkatan utama:

1. **Lapisan Hulu (Upstream Mock Services):** Terdiri atas `mock-bmkg` (port 8081) dan `mock-pvmbg` (port 8082) yang mensimulasikan penyedia data instansi eksternal secara otonom dengan karakteristik latensi dan kegagalan yang dapat dikendalikan.
2. **Lapisan Inti Koordinasi BNPB (Core Aggregation & Persistence):** Terdiri atas `aggregator` (port 8083 internal) yang memegang kepemilikan eksklusif atas basis data kanonis `canonical-db` (PostgreSQL 16, port 5432) serta mempublikasikan event kanonis ke broker Redpanda (port 9092 / 19092).
3. **Lapisan Hilir Klien & Konsumen (Downstream Client API & Event Consumers):** Terdiri atas `auth` (port 8084), `client-api` (port 8080), serta dua *consumer* independen `dashboard-consumer` dan `notifier-consumer`.

2.2 Tanggung Jawab Setiap Service

Setiap service memiliki batas tanggung jawab (*bounded context*) yang didefinisikan secara tegas tanpa adanya tumpang tindih logika:

- **`mock-bmkg`:** Mensimulasikan data gempa bumi (`SeismicEvent`) dan peringatan tsunami (`TsunamiWarning`). Menegakkan autentikasi via header `X-BMKG-Key` dan merespons dengan latensi rendah (50–150 ms).



Gambar 2.1: Diagram Arsitektur Sistem Terdistribusi BNPB Milestone 1

- **`mock-pvmbg`:** Mensimulasikan laporan gunung api (`VolcanicReport`). Menegakkan autentikasi via Bearer token, mendukung injeksi latensi tinggi (500 ms–3 s), simulasi kegagalan instan (`POST /admin/outage`), serta perubahan skema aditif saat sistem berjalan (`POST /admin/schema-version`).
- **`aggregator`:** Melakukan *ingestion polling* berkala ke kedua sumber hulu secara paralel, melakukan normalisasi ke format kanonis `HazardEvent`, memperkaya koordinat gunung api via katalog statis, menyimpan data secara transaksional ke PostgreSQL, mendeteksi gangguan hulu dan menandai status degradasi, serta mengeksekusi *Transactional Outbox Relay*.
- **`auth`:** Mengelola direktori klien, menerbitkan token akses JWT berumur 60 detik dengan klaim *scope* terisolasi, serta mengelola rotasi *refresh token* dengan deteksi *replay revocation*.
- **`client-api`:** Menyediakan gerbang HTTP publik bagi pihak hilir. Bertanggung jawab memverifikasi tanda tangan JWT dan cakupan hak akses, memfilter atribut mentah untuk pengguna Media, serta meneruskan permintaan ke Aggregator dengan propagasi `X-Correlation-ID`.
- **`dashboard-consumer`:** Berlangganan topik event kanonis dengan grup `bnpb-dashboard-group`, melakukan deduplikasi berbasis `hazard_id`, dan memperbarui metrik rekapitulasi bencana secara *real-time*.
- **`notifier-consumer`:** Berlangganan topik event kanonis dengan grup `bnpb-notifier-group`, mengevaluasi ambang batas darurat kebencanaan (gempa ≥ 6.0, tsunami SIAGA/AWAS, vulkanik AWAS), dan menerbitkan sinyal peringatan darurat.

2.3 Protokol Komunikasi Antarservice

Sistem menerapkan pemisahan yang ketat antara komunikasi sinkron dan asinkron:

- **Komunikasi Sinkron (HTTP/REST):** Digunakan untuk operasi permintaan-respons (*request-response*) berorientasi pembacaan data historis. Polling dari Aggregator ke mock upstream serta Client API ke Aggregator menggunakan HTTP/1.1 dengan payload JSON dan header `X-Correlation-ID`.
- **Komunikasi Asinkron (Kafka Protocol via Redpanda):** Digunakan untuk aliran diseminasi kejadian bencana ke banyak konsumen secara non-pemblokir (*non-blocking pub-sub*). Aggregator menerbitkan pesan terkanonisasi ke topik `bnpb.hazard-events` menggunakan `hazard_id` sebagai *message key* guna menjamin keterurutan partisi.

2.4 Kepemilikan Penyimpanan dan Batas Container

Sesuai prinsip arsitektur *microservices* dan klausul wajib spesifikasi Tugas Besar:

- **Single Ownership:** Basis data relasional `canonical-db` dimiliki secara eksklusif oleh `aggregator`. Service lain (seperti `client-api` atau konsumen) secara arsitektural dilarang terhubung langsung ke port 5432 database.
- **Isolasi Jaringan Docker:** Port basis data tidak diekspos ke jaringan publik host. Komunikasi antara `aggregator` dan `canonical-db` berlangsung di dalam jaringan privat `bnpb-internal`.
- **Zero Shared Libraries:** Seluruh service Go dikembangkan secara mandiri tanpa menggunakan pustaka *shared business logic* (`pkg/` atau `common/`) bersama. Masing-masing service memiliki berkas `go.mod` mandiri yang diorkestrasi via `go.work`.

2.5 Alur Permintaan, Data, dan Event

Alur pemrosesan data ujung-ke-ujung dirancang sebagai berikut:

1. **Pengambilan (Ingestion):** Dua goroutine poller mandiri dalam Aggregator secara berkala memanggil `mock-bmkg` dan `mock-pvmbg` setiap 2

detik dengan jendela tumpang tindih (*overlap*) 10 detik.

2. **Normalisasi & Transaksi:** Data dipetakan menjadi `HazardEvent`. Dalam satu transaksi ACID (`SaveHazardWithOutbox`), Aggregator menyimpan entitas bahaya ke tabel `hazards` sekaligus mencatat baris baru pada tabel `outbox_events`.
3. **Relay Outbox:** Goroutine latar belakang membaca baris outbox yang belum terkirim, mempublikasikannya ke Redpanda dengan header korelasi, lalu menandai baris outbox sebagai terkirim (*marked as sent*).
4. **Konsumsi & Deduplikasi:** Konsumen `dashboard` dan `notifier` menarik pesan secara paralel, memeriksa tabel deduplikasi memori, memproses logika bisnis masing-masing, dan melakukan *manual offset commit*.

2.6 Teknologi Terpilih dan Alasan Pemilihan

Tabel 2.1 merangkum tumpukan teknologi yang dipilih beserta justifikasi teknisnya:

Tabel 2.1: Rangkuman Pemilihan Teknologi Sistem Terdistribusi BNPB

Kategori	**Teknologi Terpilih**	**Alasan Pemilihan & Kesesuaian Kebutuhan**
Back-End	Go 1.22	Dukungan konkurensi bawaan (*goroutines* dan *channels*) yang sangat efisien, konsumsi memori rendah, kompilasi cepat ke biner statis, dan menyeragamkan pengembangan seluruh service.
Database	PostgreSQL 16 (JSONB)	Menjamin transaksi ACID penuh untuk pola Transactional Outbox, sekaligus menyediakan fleksibilitas kolom JSONB dengan pengindeksan GIN untuk menyimpan atribut dinamis tanpa migrasi DDL.
Komunikasi	HTTP/REST & JSON	Standar industri yang transparan, mudah di-debug, mendukung interoperabilitas universal, dan menyederhanakan penegakan kontrak OpenAPI antarservice.
Message Broker	Redpanda 24.1 (API Kafka)	Kompatibel penuh dengan ekosistem Kafka tanpa ketergantungan JVM atau Zookeeper, latensi deterministik berbasis C++/Raft, serta konsumsi sumber daya ringan dalam container.

Tabel 2.1: Rangkuman Pemilihan Teknologi Sistem Terdistribusi BNPB (Lanjutan)

Kategori	**Teknologi Terpilih**	**Alasan Pemilihan & Kesesuaian Kebutuhan**
Autentikasi	JWT HS256 & Refresh Token	Memungkinkan verifikasi klaim hak akses (*scope*) secara terdesentralisasi di sisi Client API tanpa panggilan jaringan berulang ke Auth Service, dengan rotasi berkala untuk keamanan.
Infrastruktur	Docker & Docker Compose	Menyediakan batas proses yang terisolasi sempurna bagi setiap komponen serta memungkinkan orkestrasi seluruh topologi multi-service melalui satu perintah (`make up`).
Dokumentasi	OpenAPI 3.0	Mendefinisikan kontrak antarmuka API yang ketat dan terstandarisasi untuk mencegah pelanggaran skema data antartim.
Pengujian	k6, curl, dan jq	k6 memungkinkan pengujian beban konkurensi tinggi ($\geq$ 50 VUs) dengan pelaporan terstruktur; curl dan jq mempermudah otomatisasi skrip demonstrasi.
Orkestrasi	GNU Make & Fish/Bash	Menyeragamkan eksekusi skrip demonstrasi per problem (`make demo-p1` hingga `demo-p5`) agar pengujian dapat direproduksi secara instan.

Bab 3

Solusi dan Pembahasan Permasalahan

3.1 Problem 1: Arsitektur dan Interoperabilitas Data

3.1.1 Rancangan Solusi dan Mekanisme

Permasalahan pertama (P1) berkaitan dengan kerapuhan sistem terhadap perubahan skema data dari instansi hulu yang terjadi secara dinamis tanpa koordinasi terlebih dahulu. Pada kasus ini, PVMBG memperkenalkan atribut baru berupa `confidence_level` (nilai float 0.0–1.0) di tengah masa operasional sistem.

Untuk mengatasi permasalahan tersebut tanpa mengharuskan *redeploy* atau modifikasi kode pada Aggregator, sistem menerapkan mekanisme **Hybrid Schemaless Ingestion & Dynamic Attribute Mapping**:

1. **Penangkapan Skema Dinamis di Adapter:** Struktur data `VolcanicReport` pada adapter Aggregator memisahkan atribut wajib yang telah disepakati dari atribut yang tidak terpetakan dengan memanfaatkan pemetaan `DynamicFields map[string]any`. Serialisasi JSON dilakukan secara fleksibel sehingga setiap kunci yang tidak terdaftar dalam struct Go tidak dibuang atau menyebabkan galat parsing.
2. **Penyeragaman ke Skema Kanonis:** Data diubah ke dalam bentuk `HazardEvent`. Atribut standar yang memiliki korespondensi langsung (seperti `report_id`, `alert_level`, `reported_at`) dipetakan ke kolom-kolom relasional kanonis terindeks. Atribut spesifik domain yang tidak

memiliki padanan langsung (termasuk `ash_column_height_m`, `eruption_count_24h`, dan field baru `confidence_level`) secara otomatis dialirkan ke dalam map `attributes`.

3. **Persistensi Tanpa Migrasi DDL:** Nilai `attributes` disimpan ke dalam kolom PostgreSQL bertipe `JSONB`. Dengan pendekatan ini, penambahan field baru dari hulu tidak memerlukan eksekusi `ALTER TABLE` maupun penguncian tabel (*table lock*).

3.1.2 Alasan Keputusan dan Trade-Off

Dalam menentukan mekanisme evolusi skema, kelompok mempertimbangkan beberapa alternatif:

- **Schema Registry Terpusat (misal: Confluent Schema Registry):** Alternatif ini menjamin validasi skema yang sangat ketat, tetapi memperkenalkan ketergantungan infrastruktur tambahan serta memerlukan sinkronisasi aktif dari instansi hulu yang tidak sejalan dengan premis bahwa hulu beroperasi secara otonom dan pasif.
- **Schema Evolution Berbasis Binary (Protobuf / Avro):** Mendukung evolusi berbasis nomor penanda field, namun tidak fleksibel terhadap respons REST/JSON dari mock eksternal yang heterogen.
- **Dynamic JSON Mapping ke PostgreSQL JSONB (Solusi Terpilih):** Memberikan fleksibilitas maksimal terhadap penambahan atribut baru secara aditif tanpa *downtime*, dengan *trade-off* validasi tipe data pada atribut dinamis harus ditangani secara defensif di lapisan aplikasi.

3.1.3 Batasan yang Disadari

Mekanisme yang diimplementasikan memiliki batasan arsitektur yang disadari:

- Sistem hanya mendukung **perubahan skema yang bersifat aditif** (*additive-only changes*), yaitu penambahan atribut baru atau atribut opsional.
- Sistem secara sadar **tidak mendukung perubahan yang merusak** (*breaking changes*), seperti penghapusan atribut kunci identitas primer (`report_id`), penggantian nama field esensial (*renaming*), atau perubahan tipe data primitif pada atribut inti.

3.1.4 Bukti Penyelesaian

Verifikasi empiris dijalankan melalui skrip pengujian otomatis `scripts/test_schema_change.sh` (`make demo-p1`) dengan hasil eksekusi tercatat pada berkas log `docs/evidence/p1/demo_p1.log`:

1. **Fase Awal (Baseline Schema):** PVMBG beroperasi dengan parameter `include_confidence_level = false`. Aggregator berhasil menghasilkan record kanonis tanpa field `confidence_level`.
2. **Pemicuan Perubahan Skema Dinamis:** Mengirimkan permintaan `POST /admin/schema-version` dengan payload `{"include_confidence_level": true}` ke `mock-pvmbg` (:8082). Upstream segera memancarkan field baru.
3. **Verifikasi Ingesti & Client API:** Tanpa mematikan atau me-restart container mana pun (`mock-pvmbg`, `aggregator`, `client-api`), Aggregator langsung mengonsumsi record baru dan menempatkan `confidence_level` ke dalam `attributes JSONB`:

```
{
  "hazard_id": "haz-pvmbg-pvmbg-rpt-live-20261007-144707-000003",
  "source": "PVMBG",
  "hazard_type": "VOLCANIC",
  "severity": "SIAGA",
  "area_name": "Gunung Semeru",
  "attributes": {
    "ash_column_height_m": 450,
    "confidence_level": 0.82,
    "eruption_count_24h": 3,
    "volcano_id": "VLC-003"
  }
}
```

Listing 3.1: Cuplikan Respons Client API Menampilkan Atribut Dinamis confidence_level

3.1.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Deklarasikan kebijakan penanganan field tak dikenal di laporan (diteruskan ke attributes atau diabaikan) serta nyatakan eksplisit jenis perubahan skema yang didukung dan yang tidak didukung.*

Jawaban:

1. **Kebijakan Field Tak Dikenal (Unknown Field Policy):** Kelompok secara eksplisit mendeklarasikan kebijakan **Forward to Canonical Attributes (*Preserve & Ingest*)**. Setiap field baru yang tidak dikenali dalam skema relasional inti tidak akan diabaikan atau dibuang, melainkan diteruskan seutuhnya ke dalam kolom `attributes JSONB` agar data domain bernilai tinggi tidak hilang.

2. **Perubahan yang Didukung:** Penambahan atribut baru bertipe primitif (*number*, *string*, *boolean*) atau bersarang (*nested object/array*) pada respons JSON instansi sumber.
3. **Perubahan yang Tidak Didukung:** Perubahan destruktif yang mencakup penghapusan field penentu identitas (`event_id`, `report_id`), pengubahan format serialisasi (misal: JSON menjadi XML), atau perubahan tipe data field esensial.

3.2 Problem 2: Concurrency, Availability, dan Graceful Degradation

3.2.1 Rancangan Solusi dan Mekanisme

Permasalahan kedua (P2) berfokus pada pencegahan tiga titik kegagalan utama dalam sistem terdistribusi ketika berhadapan dengan beban tinggi dan gangguan penyedia data eksternal:

1. **Pencegahan Head-of-Line Blocking:** Aggregator mengisolasi jalur pemanggilan BMKG dan PVMBG ke dalam *goroutines worker* yang beroperasi secara independen dan asinkron. Poller BMKG dan PVMBG menggunakan klien HTTP dengan batas waktu (*timeout*) mandiri (2 detik untuk BMKG, 5 detik untuk PVMBG). Kelambatan ekstrem atau penundaan sintetis (3 detik) pada PVMBG sama sekali tidak menahan proses penarikan atau penyajian data seismik BMKG.
2. **Pencegahan Resource Exhaustion (Kelelahan Sumber Daya):** Pada lapisan HTTP publik (`client-api`), batas konkurensi dikendalikan melalui penyangga koneksi HTTP Go dan konfigurasi `pgxpool` pada Aggregator (`MaxConns = 50`, `MinConns = 10`, `MaxConnLifetime = 1h`). Dengan pembatasan ini, lonjakan permintaan konkuren tidak mengakibatkan kebocoran soket atau kehabisan memori.
3. **Penerapan Graceful Degradation (Degradasi Terkendali):** Aggregator mengimplementasikan pelacak keandalan (*Health & Degradation Tracker*) berbasis pola *Circuit Breaker*. Jika PVMBG mengalami kegagalan beruntun (*consecutive failures* ≥ 3) atau mengembalikan HTTP 503, Aggregator secara otomatis mengaktifkan status degradasi (`is_degraded = true`) dan mencatat stempel waktu `stale_since`. Saat klien hilir meminta data gabungan, sistem tetap mengembalikan respons HTTP 200 berisi data BMKG terbaru dan data historis terakhir PVM-

BG dari penyimpanan kanonis, disertai metadata penanda eksplisit.

3.2.2 Alasan Keputusan dan Trade-Off

Dalam menangani konkurensi dan degradasi, kelompok mengevaluasi trade-off berikut:

- **Pola Fail-Fast (HTTP 503 Langsung ke Klien):** Menolak permintaan klien secara instan saat salah satu sumber mati. Pilihan ini ditolak karena menghilangkan kegunaan data bencana bagi tim lapangan saat mereka sangat membutuhkan data seismik BMKG meskipun data vulkanik terganggu.
- **Stale-Data Fallback dengan Penanda Kebaruan (Solusi Terpilih):** Sistem mengutamakan ketersediaan (*Availability* di atas konsistensi mutlak per teorema CAP). Klien menerima data yang ada dengan transparansi status melalui field `meta.degraded = true` dan `meta.stale_since`.

3.2.3 Batasan yang Disadari

- Pada kondisi degradasi berkepanjangan, data vulkanik yang disajikan semakin usang (*stale*). Sistem Milestone 1 mengandalkan pemahaman klien terhadap atribut `meta.stale_since` dan belum mengimplementasikan estimasi interpolasi bencana sintetis.
- Pembatasan laju permintaan (*rate limiting*) pada Milestone 1 ditangani pada level kapasitas koneksi server (*connection backlog*), belum menggunakan token bucket terdistribusi Redis.

3.2.4 Bukti Penyelesaian

Verifikasi empiris dilakukan melalui dua skrip utama:

1. **Uji Isolasi Latensi & Simulasi Outage (`make demo-p2`):** PVMBG dimatikan melalui `POST /admin/outage` (`is_outage = true`). Klien API tetap melayani permintaan dengan status HTTP 200 dan menghasilkan payload terdegradasi:

```
{
  "data": [ ... ],
  "meta": {
    "total": 1562,
    "limit": 5,
    "offset": 0,
    "degraded": true,
    "stale_since": "2026-10-08T09:45:21.5830728Z"
```

```
  }
}
```

Listing 3.2: Respons Klien API dalam Keadaan Terdegradasi (meta.degraded = true)

Sementara itu, pemanggilan langsung ke `mock-bmkg` (:8081) tetap merespons dengan HTTP 200 dan latensi p95 jauh di bawah ambang 300 ms. Log eksekusi tersimpan pada `docs/evidence/p2/demo_p2.log`.

2. **Uji Beban Konkurensi Berkelanjutan (`make load-test`):** Pengujian beban menggunakan k6 dijalankan dengan parameter 50 Virtual Users (VUs) secara simultan selama 60 detik penuh terhadap titik akhir `GET http://localhost:8080/api/v1/hazards`. Hasil eksekusi tercatat pada `docs/evidence/p2/load_test.log`:
   - **Total Permintaan Terlayani:** 29.514 permintaan ($\approx$ 491 req/s).
   - **Latensi p95:** 1,75 ms (Ambang batas spesifikasi: $<$ 300 ms) — **MEMENUHI**.
   - **Latensi p99:** 3,07 ms (Ambang batas spesifikasi: $<$ 500 ms) — **MEMENUHI**.
   - **Error Rate:** 0,43% (Ambang batas spesifikasi: $< 1,0\%$) — **MEMENUHI**.

3.2.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan model konkurensi yang dipilih dan bagaimana model tersebut mencegah ketiga kegagalan di atas. Sertakan konfigurasi load testing beserta tooling yang dipakai.*

Jawaban:

1. **Model Konkurensi Terpilih:** Sistem mengadopsi model konkurensi berbasis *Communicating Sequential Processes (CSP)* yang diwujudkan melalui *goroutines* dan *buffered channels* di Go:
   - *Head-of-line blocking* dicegah dengan memisahkan goroutine penarik BMKG dan PVMBG secara asinkron; penundaan pada satu socket TCP upstream tidak menghambat pemrosesan socket lainnya.
   - *Resource exhaustion* dicegah melalui pembatasan ukuran *connection pool* PostgreSQL (`pgxpool.Config.MaxConns = 50`) dan pemanfaatan konkurensi ringan goroutine (hanya memakan memori $\approx$ 2 KB per goroutine dibanding utas OS standar 2–8 MB).

- *Ketiadaan degradasi terkendali* dicegah dengan pencatatan state kesehatan penyedia data; saat kegagalan terdeteksi, Aggregator beralih menyajikan snapshot terakhir basis data kanonis dengan flag `meta.degraded = true`.

2. **Konfigurasi Load Testing:**
   - **Tooling:** Grafana k6 v0.50+ dieksekusi secara otomatis melalui Docker atau biner lokal (`scripts/load_test.js`).
   - **Beban & Durasi:** 50 Virtual Users (VUs) selama 60 detik konstan (*sustained load*).
   - **Autentikasi:** Setiap VU melakukan autentikasi otomatis ke Auth Service dan menyertakan header `Authorization: Bearer <token>` pada setiap iterasi.

3.3 Problem 3: Pengendalian Hak Akses dan Isolasi Kredensial

3.3.1 Rancangan Solusi dan Mekanisme

Permasalahan ketiga (P3) berkaitan dengan pemisahan domain kepercayaan (*domain of trust*). Jika satu kredensial bocor (misalnya kunci yang dipegang oleh pihak Media), penyerang tidak boleh dapat menggunakannya untuk menembus batasan akses data mentah pada API BNPB maupun menggunakannya langsung ke API sumber BMKG atau PVMBG.

Untuk menyelesaikan permasalahan ini, sistem menerapkan arsitektur isolasi berlapis:

1. **Isolasi Kredensial Hulu:** Layanan BMKG mewajibkan header `X-BMKG-Key: bmkg-secret-key-2026`, sedangkan PVMBG mewajibkan header `Authorization: Bearer <pvmbg-bearer-token-2026>`. Format dan tanda tangan yang berbeda ini memaksa penolakan instan (HTTP 401/403) jika kredensial dipertukarkan (*cross-credential reuse*).
2. **Layanan Autentikasi Terpusat (Auth Service):** Menerbitkan token akses JWT bertanda tangan HS256 (`POST /auth/token`) dengan masa berlaku (*TTL*) pendek, yaitu 60 detik secara default. Klien memohon token berdasarkan peran yang sah (`media`, `field`, `internal`) yang dipetakan ke klaim `scope`:
   - Klien `media` → klaim scope `hazards:read:summary`.

- Klien `field` dan `internal` → klaim scope `hazards:read:all`.

3. **Penegakan Hak Akses di Sisi Server (Client API):** Client API memvalidasi tanda tangan kriptografi JWT, masa kedaluwarsa, dan klaim `scope`. Klien Media hanya menerima field ringkasan. Jika klien Media secara eksplisit meminta field mentah (`?fields=all`), server menolak dengan HTTP 403 Forbidden.
4. **Rotasi Refresh Token Sekali Pakai (Single-Use Token Rotation):** Klien Tim Lapangan memperoleh *refresh token* kriptografis acak 32-byte. Saat token akses kedaluwarsa setelah 60 detik, klien memanggil `POST /auth/refresh`. Auth Service memvalidasi refresh token, mencabut (*burn*) token lama, dan menerbitkan pasangan token baru. Jika token lama digunakan kembali (*replay attack*), sistem secara otomatis membatalkan seluruh sesi refresh terkait.

3.3.2 Alasan Keputusan dan Trade-Off

- **JWT Terdesentralisasi vs Token Referensi Terpusat:** Verifikasi JWT secara lokal di Client API mengeliminasi latensi jaringan panggilan balik (*network hop*) ke Auth Service pada setiap permintaan data, dengan *trade-off* token akses yang telah terbit tidak dapat dicabut seketika sebelum masa berlakunya habis (dimitigasi dengan TTL pendek 60 detik).
- **Kredensial Otonom Antar-Instansi:** Meniadakan penggunaan kunci master tunggal, menjamin prinsip otonomi dan batas keamanan antarlembaga.

3.3.3 Batasan yang Disadari

- Hak akses pada Milestone 1 dibatasi pada tiga kategori klien dengan hak baca (*read-only*).
- Sesi *refresh token* pada Auth Service disimpan dalam memori lokal proses (*in-memory map*), sehingga sesi klien akan tereset apabila container Auth di-restart.

3.3.4 Bukti Penyelesaian

Verifikasi diotomasi melalui `make demo-p3` yang mengeksekusi `scripts/test_cross_credential.sh` dan `scripts/test_auth_scopes.sh`:

1. **Uji Penolakan Lintas Kredensial:** Kunci BMKG yang dikirimkan ke PVMBG menghasilkan HTTP 401 Unauthorized. Sebaliknya, token PVMBG yang dikirimkan ke BMKG menghasilkan HTTP 401 Unauthorized.
2. **Uji Penegakan RBAC Media:** Klien Media berhasil mengakses data ringkasan (`200 OK`), namun saat mengirimkan `GET /api/v1/hazards?fields=all`, Client API secara konsisten mengembalikan `403 Forbidden`.
3. **Uji Kedaluwarsa Alami 60 Detik & Rotasi Refresh:** Token akses diverifikasi berhasil pada detik ke-0. Setelah menunggu 62 detik tanpa memanipulasi jam sistem, token lama ditolak (`401 Unauthorized`). Pemanggilan `/auth/refresh` berhasil menerbitkan token baru. Replay token lama langsung ditolak. Seluruh 15 skenario pengujian berhasil 100%.

3.3.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan mekanisme yang dipakai beserta alasan mengapa mekanisme tersebut tidak membuka celah baru.*

Jawaban: Mekanisme yang diterapkan menjamin keamanan tanpa memperkenalkan celah baru karena:

1. **Imunitas Terhadap Kebocoran Klien Media:** Klien Media tidak pernah menerima kredensial upstream (BMKG/PVMBG) maupun kunci rahasia HMAC. Klien hanya memegang JWT dengan klaim `scope: hazards:read:summary`. Jika token media bocor, penyerang tetap tidak dapat mengakses endpoint mentah karena Client API memvalidasi scope secara ketat di server, dan token tersebut ditolak mentah-mentah jika diarahkan ke API BMKG atau PVMBG.
2. **Pencegahan Pemalsuan Token (Signature Verification):** JWT ditandatangani menggunakan algoritma HS256 dengan `JWT_SECRET` yang hanya diketahui oleh Auth Service dan Client API via environment variable. Klien tidak dapat merekayasa scope dari `summary` menjadi `all` tanpa merusak signature.
3. **Perlindungan Terhadap Pencurian Refresh Token:** Setiap refresh token hanya valid untuk satu kali pemakaian (*single-use*). Upaya penyerang menggunakan refresh token yang telah terpakai memicu deteksi *replay revocation* yang seketika menghanguskan seluruh silsilah sesi refresh tersebut.

3.4 Problem 4: Arsitektur Microservice dan Penyimpanan Fleksibel

3.4.1 Rancangan Solusi dan Mekanisme

Permasalahan keempat (P4) menyasar dua kelemahan sistem monolitik terikat: kegagalan isolasi saat pembangunan ulang (*rebuild*) serta kekakuan skema basis data relasional klasik yang memaksa migrasi DDL pada setiap evolusi atribut hulu.

Solusi dirancang melalui integrasi dua pilar arsitektur:

1. **Batas Layanan Mandiri (*Service Decoupling*):** Setiap komponen dibungkus dalam container mandiri dengan siklus hidup terisolasi. Pembangunan ulang atau *restart* pada container `aggregator` atau `canonical-db` tidak memicu kegagalan berantai (*cascading failure*) pada `client-api`, `mock-bmkg`, maupun `mock-pvmbg`. Layanan hilir dilengkapi mekanisme *graceful reconnect* otomatis terhadap koneksi basis data yang terputus sementara.
2. **Penyimpanan Kanonis Fleksibel (Hybrid Relational + JSONB):** Skema tabel `hazards` memadukan kolom relasional bertipe kuat untuk dimensi pencarian umum (`hazard_id`, `source`, `hazard_type`, `severity`, `occurred_at`) dengan kolom `attributes JSONB` untuk atribut spesifik instansi. Ketika PVMBG memperkenalkan field `confidence_level`, data langsung disimpan berdampingan dengan data lama tanpa memerlukan instruksi `ALTER TABLE` maupun penguncian tabel.
3. **Kepemilikan Tunggal (*Single Ownership*):** Port 5432 database hanya terhubung ke Aggregator di jaringan internal. Client API dilarang keras melakukan koneksi SQL langsung, melainkan mengambil data melalui antarmuka HTTP internal Aggregator (`:8083/internal/hazards`).

3.4.2 Alasan Keputusan dan Trade-Off

Tabel 3.1 menyajikan perbandingan komparatif tiga opsi arsitektur penyimpanan:

3.4.3 Batasan yang Disadari

- Kolom JSONB tidak menegakkan validasi tipe data pada level basis data. Integritas tipe nilai dalam atribut dinamis sepenuhnya menjadi tanggung

Tabel 3.1: Evaluasi Komparatif Alternatif Arsitektur Penyimpanan

Kriteria	**PostgreSQL + JSONB (Terpilih)**	**MongoDB (Document Store)**	**Strict SQL (EAV / Migrasi DDL)**
Transaksionalitas & Pola Outbox	**Sangat Kuat:** Mendukung transaksi ACID atomik antara tabel `hazards` dan `outbox_events`.	**Kompleks:** Transaksi multi-dokumen memerlukan replika set terdistribusi dengan penalti performa.	**Kuat:** Mendukung transaksi ACID penuh.
Agilitas Evolusi Skema	**Tinggi:** Atribut baru masuk ke JSONB tanpa *table lock* atau migrasi DDL.	**Tinggi:** Skema dokumen secara alami dinamis.	**Rendah:** Memerlukan DDL `ALTER TABLE` atau tabel EAV yang rumit.
Performa Query & Indexing	**Tinggi:** Kolom relasional diindeks B-Tree standar; kolom JSONB dapat diindeks GIN.	**Sedang:** Kueri dokumen sangat cepat, namun tidak mendukung relasi JOIN.	**Sedang–Rendah:** Pola EAV memperlambat kueri karena banyaknya operasi JOIN.
Kompleksitas Operasional	**Rendah:** Satu mesin basis data melayani kebutuhan relasional dan dokumen fleksibel.	**Sedang:** Menambah beban pemeliharaan infrastruktur klaster terpisah.	**Rendah:** Mesin relasional tunggal namun beban migrasi skema tinggi.

jawab lapisan kode normalizer di Aggregator.

- Kueri berbasis teks bebas atau pencarian parsial di dalam dokumen JSONB yang belum diindeks GIN memerlukan *full-table scan* parsial jika tabel bertumbuh hingga jutaan baris.

3.4.4 Bukti Penyelesaian

Verifikasi diotomasi melalui `make demo-p4` (`scripts/test_storage_restart.sh`) dengan rekaman log tersimpan di `docs/evidence/p4/storage_restart.log`:

1. **Koeksistensi Skema Heterogen pada PostgreSQL (P4-02):** Kueri langsung ke basis data `canonical-db` membuktikan record sebelum evolusi skema dan record setelah evolusi skema hidup berdampingan secara harmonis:

```
SELECT hazard_id, source, hazard_type, severity, attributes
FROM hazards ORDER BY occurred_at DESC LIMIT 2;

-- Record 1 (Evolved PVMBG):
-- attributes: {"volcano_id": "VLC-002", "confidence_level": 0.82, "
    ash_column_height_m": 350}
```

```
-- Record 2 (Baseline BMKG):
-- attributes: {"depth_km": 36, "magnitude": 4.9, "potential_tsunami":
    true}
```

Listing 3.3: Hasil Kueri PostgreSQL Menunjukkan Koeksistensi Atribut Heterogen

2. **Uji Restart Kontainer Independen (P4-01):** Container `aggregator` di-restart via Docker Compose. Selama proses restart, `client-api` (:8080) dan mock services tetap merespons dengan HTTP 200 tanpa mengalami kegagalan berantai.
3. **Pemulihan Mandiri Sambungan Basis Data:** Container `canonical-db` di-restart. Klien API mendeteksi ketiadaan basis data sementara dan seketika pulih melayani permintaan dengan HTTP 200 begitu PostgreSQL siap menerima koneksi kembali.

3.4.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan alasan penentuan batas tiap service, mengapa dipecah seperti itu dan bukan dengan pembagian lain. Serta, bandingkan minimal dua alternatif penyimpanan beserta konsekuensi dari pilihan yang diambil.*

Jawaban:

1. **Alasan Penentuan Batas Service (*Service Boundaries*):** Service dipecah berdasarkan batas domain dan tingkat perubahan (*rate of change*):
   - *Aggregator* dipisahkan karena memiliki dependensi polling aktif dan merupakan satu-satunya penulis (*single writer*) ke penyimpanan kanonis.
   - *Client-API* dipisahkan agar dapat diskalakan secara horizontal sesuai lonjakan lalu lintas baca publik tanpa membebani thread penarik data ke hulu.
   - *Auth Service* dipisahkan guna mengisolasi rahasia tanda tangan token kriptografi serta mencegah beban verifikasi autentikasi mengganggu pipa ingesti darurat.
2. **Perbandingan Alternatif Penyimpanan:** Dua alternatif utama dievaluasi (Tabel 3.1):
   - *Alternatif 1: Basis Data Dokumen Murni (MongoDB):* Keuntungannya fleksibel secara alami, namun konsekuensi buruknya adalah implementasi Transactional Outbox memerlukan transaksi multi-

dokumen terdistribusi yang sangat kompleks dan rentan performa.

- *Pilihan Terpilih: PostgreSQL 16 dengan Kolom JSONB:* Konsekuensinya adalah kelompok harus menjaga validasi skema atribut dinamis di lapisan aplikasi, namun memperoleh keuntungan besar berupa transaksi ACID tunggal atomik untuk outbox event, pengindeksan relasional yang matang, serta keandalan operasional tingkat tinggi.

3.5 Problem 5: Fan-out Event Distribution atau Publish–Subscribe

3.5.1 Rancangan Solusi dan Mekanisme

Pada sistem agregasi bencana multi-lembaga, volume dan ragam pemangku kepentingan hilir terus bertambah seiring waktu. Kebutuhan tim operasional internal mencakup pembaruan metrik visual secara instan pada *dashboard*, tim kesiapsiagaan darurat membutuhkan pengiriman notifikasi peringatan dini (*early warning*) secara langsung ke regu penanggulangan bencana di lapangan, dan instansi eksternal (seperti portal pemerintah daerah/Pemda) berencana menyerap aliran data bencana yang sama.

Apabila *Aggregator Service* memanggil seluruh sistem hilir secara sinkron melalui HTTP/REST langsung (*point-to-point synchronous chaining*), muncul permasalahan *cascading latency* dan *tight coupling*: keterlambatan pada satu sistem hilir (misalnya koneksi notifikasi seluler yang lambat) akan menahan siklus eksekusi *Aggregator* dan menunda pengiriman data ke sistem lain yang lebih mendesak. Selain itu, setiap penambahan konsumen baru akan mewajibkan modifikasi kode dan *redeploy* pada *Aggregator*.

Untuk mengatasi permasalahan tersebut, perancangan sistem menerapkan pola arsitektur ***Publish–Subscribe*** berbasis *log-based message broker* menggunakan Redpanda (kompatibel penuh dengan protokol Apache Kafka). Rancangan dan mekanisme solusi adalah sebagai berikut:

1. **Decoupled Event Publishing:** Setelah *Aggregator Service* menerima, memetakan, dan memvalidasi rekaman bencana dari lembaga sumber, *Aggregator* menerbitkan entitas kanonikal `HazardEvent` ke *topic* `bnpb.hazard-events` pada Redpanda. *Topic* ini dikonfigurasi dengan 3 partisi dan memanfaatkan `hazard_id` sebagai *partition key* untuk menjamin *causal ordering* per kejadian bencana. Penerbitan dilakukan secara

asinkron tanpa keterikatan terhadap identitas, alamat IP, maupun ketersediaan konsumen mana pun.

2. **Independent Consumer Groups:** Setiap layanan konsumen hilir beroperasi di bawah *consumer group* yang terisolasi secara independen:
   - `bnpb-dashboard-group`: Dikelola oleh *Dashboard Consumer* untuk mengagregasi statistik frekuensi jenis bencana dan tingkat keparahan.
   - `bnpb-notifier-group`: Dikelola oleh *Notifier Consumer* untuk mengirimkan notifikasi simulasi peringatan tingkat *Waspada*, *Siaga*, dan *Awas*.
   - `bnpb-evaluator-audit-group` / Konsumen Eksternal: Konsumen ketiga (misalnya audit atau portal pemda) yang dapat mendaftar dan membaca aliran data secara independen.

   Masing-masing kelompok konsumen mengelola posisi penunjuk pembacaan (*committed offset*) secara terpisah pada metadata Redpanda (`__consumer_offsets`). Kecepatan konsumsi maupun gangguan pada satu grup tidak berdampak sama sekali terhadap grup lainnya.
3. **Consumer Downtime Resilience dan Backlog Recovery:** Apabila suatu layanan konsumen (misalnya *Dashboard*) mengalami kegagalan proses atau dihentikan (*downtime*), Redpanda tetap menyimpan pesan-pesan baru yang diterbitkan produser ke dalam *append-only log* pada disk broker. Nilai *consumer lag* untuk grup tersebut akan bertambah secara proporsional. Sementara itu, konsumen lain (*Notifier*) tetap memproses pesan baru secara *real-time* tanpa terganggu. Ketika layanan yang mati dihidupkan kembali, konsumen membaca ulang konfigurasi *committed offset* terakhirnya dan segera mengejar ketertinggalan (*catch-up backlog*) hingga nilai *lag* kembali ke nol.
4. **Manual Offset Commit Discipline:** Konsumen mengimplementasikan pustaka `kafka-go` dengan pembacaan berbasis `FetchMessage`. *Offset* pesan tidak dikomit secara otomatis, melainkan secara manual melalui `CommitMessages` tepat setelah pemrosesan data internal dan deduplikasi lokal berhasil diselesaikan. Hal ini menjamin semantik pengiriman *at-least-once*.

3.5.2 Alasan Keputusan dan Trade-Off

Pemilihan pola dan teknologi distribusi *event* didasarkan pada perbandingan arsitektur berikut:

1. **Pola Publish–Subscribe vs. Pola Polling Sinkron / Webhook:**
    - *Polling Terjadwal:* Jika konsumen secara berkala melakukan HTTP GET ke *Aggregator*, timbul beban kueri basis data berulang (*database thrashing*) dan latensi deteksi bencana menjadi tinggi tergantung interval *polling*.
    - *Direct Webhook/HTTP Calls:* Membebani *Aggregator* dengan tanggung jawab *retry*, manajemen kegagalan koneksi, *timeout handling*, dan registrasi dinamis URL konsumen.
    - *Keputusan:* Pola *Publish–Subscribe* dipilih karena memisahkan tanggung jawab secara bersih (*separation of concerns*). Produser berfokus pada validasi dan penerbitan *event*, sedangkan broker menangani retensi, *buffering*, partisi, dan distribusi data.
2. **Pemilihan Redpanda (Log-based Pub/Sub) vs. Tradisional Message Queue (RabbitMQ AMQP):**
    - *RabbitMQ (AMQP Fanout Exchange):* Menyediakan mekanisme distribusi pesan yang baik, namun model antreannya menghapus pesan setelah diakui (*destructive read*). Apabila konsumen baru mendaftar di kemudian hari atau konsumen yang mati membutuhkan pemutaran ulang (*replayability*) dari *offset* awal hari, antrean AMQP membutuhkan konfigurasi antrean terpisah sebelum pesan diterbitkan.
    - *Redpanda (Kafka API):* Menggunakan model *immutable distributed commit log*. Pesan dipertahankan berdasarkan durasi retensi waktu dan kuota penyimpanan fisik, bukan status pembacaan konsumen. Konsumen baru dapat bergabung kapan saja dan membaca data tanpa intervensi produser. Redpanda dipilih karena diimplementasikan dalam C++ tanpa ketergantungan JVM atau Zookeeper, berkinerja tinggi, dan hemat memori pada lingkungan kontainer Docker.

Trade-Off Arsitektural:

- **Kelebihan:** *Zero coupling* antara produser dan konsumen; latensi *end-to-end* sub-milidetik untuk *fan-out*; ketahanan data tinggi terhadap ke-

gagalan komponen hilir (*fault isolation*); penambahan konsumen baru sepenuhnya bersifat non-intrusif.

- **Konsekuensi Negatif:** Diperlukan pengelolaan infrastruktur broker pesan tambahan (*cluster health*, partisi, dan kuota disk); konsistensi data berubah menjadi *eventual consistency* (sistem hilir memiliki selisih latensi beberapa milidetik dibandingkan basis data *Aggregator*); konsumen wajib menangani potensi duplikasi pesan akibat semantik *at-least-once*.

3.5.3 Batasan yang Disadari

Batasan teknis yang disadari dan sengaja dibatasi pada lingkup implementasi saat ini meliputi:

1. **Deduplikasi Dalam Memori Bersifat Ephemeral:** Pada implementasi prototipe *Dashboard* dan *Notifier*, deduplikasi `hazard_id` disimpan dalam struktur *hash map* berbasis memori (`sync.Mutex` di Go). Apabila kontainer konsumen dihentikan secara paksa atau di-*restart*, riwayat kunci deduplikasi dalam memori akan tereset.
2. **Edge Case At-Least-Once pada Crash Antara Eksekusi dan Commit:** Apabila konsumen mengalami kegagalan sistem (*panic* atau *SIGKILL*) tepat setelah efek samping dijalankan (misalnya mencetak log agregasi) tetapi sebelum fungsi `reader.CommitMessages()` dieksekusi, pesan yang sama akan dikonsumsi kembali setelah kontainer hidup kembali.
3. **Batas Retensi Log Broker:** Penyimpanan pesan pada Redpanda dibatasi oleh kebijakan retensi waktu dan ukuran log. Jika konsumen mengalami *downtime* melebihi durasi retensi log yang dikonfigurasi, pesan-pesan terlama yang belum terbaca akan terhapus secara otomatis oleh broker sehingga konsumen tidak dapat lagi memulihkan data tersebut secara utuh.

3.5.4 Bukti Penyelesaian

Pengujian dan validasi solusi Problem 5 dilakukan secara otomatis melalui skrip `scripts/test_consumer_recovery.sh` (dapat dijalankan melalui `make demo-p5`). Seluruh pengujian mencakup verifikasi partisi, penerbitan paralel, pemulihan *downtime*, dan penambahan konsumen dinamis.

```
1 ======================================================================
2    Problem 5 Demo: Event Streaming, Downtime Recovery & 3rd Consumer
```

```
======================================================================

[Step 0] Checking Redpanda and the configured topic...
[PASS] Redpanda is responsive.
PARTITIONS: 3
[PASS] Topic 'bnpb.hazard-events' has 3 partitions.

[Step 1] Starting the Dashboard and Notifier consumers...
[PASS] Dashboard and Notifier containers are running.

[Step 2] Publishing one event for both independent consumer groups...
[PASS] Dashboard received event 'c08-fanout-1760021884-21948'.
[PASS] Notifier received the same event 'c08-fanout-1760021884-21948'.

[Step 3] Stopping Dashboard and publishing an event during downtime...
[PASS] Notifier continued processing while Dashboard was stopped.
[PASS] The recovery event remained pending for the stopped Dashboard group.
GROUP                TOPIC               STATUS    TOTAL-LAG
bnpb-dashboard-group  bnpb.hazard-events  Empty     1
[PASS] Dashboard group accumulated lag while offline.

[Step 4] Restarting Dashboard and verifying backlog recovery...
[PASS] Dashboard resumed from its committed offset and processed the missed
    event.
GROUP                TOPIC               STATUS    TOTAL-LAG
bnpb-dashboard-group  bnpb.hazard-events  Stable    0
[PASS] Dashboard consumer-group lag returned to zero.

[Step 5] Adding a dynamic 3rd consumer group...
[PASS] A new 3rd consumer group received events without publisher changes.

Results: 9 passed, 0 failed
======================================================================
   Problem 5 Verification Complete: Fan-Out & Recovery Verified!
======================================================================
```

Listing 3.4: Keluaran Eksekusi Pengujian Otomatis Problem 5 (Fan-Out & Downtime Recovery)

Analisis Bukti Log:

1. **Fan-Out Verifikasi (Langkah 2):** Satu kejadian bencana (`c08-fanout-...`) diterbitkan ke Redpanda. Kedua layanan (*Dashboard* dan *Notifier*) mendeteksi dan memproses pesan tersebut secara simultan dalam kelompok konsumen masing-masing (`bnpb-dashboard-group` dan `bnpb-notifier-group`).
2. **Downtime & Lag Accumulation (Langkah 3):** Saat kontainer *Dashboard* dimatikan paksa, pengiriman *event* baru (`c08-recovery-...`) tetap diproses normal oleh *Notifier*. Pemeriksaan Redpanda melalui perintah `rpk group describe` menunjukkan `TOTAL-LAG = 1` pada *Dashboard*, membuktikan pesan aman tersimpan pada broker dan tidak hilang.
3. **Backlog Catch-Up (Langkah 4):** Begitu kontainer *Dashboard* dinyalakan kembali, konsumen segera menyerap pesan yang tertinggal dan melakukan *manual commit*, ditandai dengan `TOTAL-LAG` yang kembali menjadi 0.
4. **Penambahan Konsumen Baru (Langkah 5):** Kontainer independen ketiga (`bnpb-evaluator-audit-...`) dijalankan pada *topic* `bnpb.hazard-events`

tanpa melakukan *restart* atau pengubahan kode apa pun pada *Aggregator Service*. Konsumen baru langsung menerima *event* secara transparan.

3.5.5 Jawaban Butir "Di Laporan"

Sesuai dengan butir penugasan pada dokumen spesifikasi resmi (*Problem 5: Fan-out Event Distribution/Pub-sub*), berikut adalah jawaban dan penjelasan komprehensif atas aspek yang diwajibkan:

1. **Broker dan Pola yang Dipilih:**
   - *Broker:* Dipilih **Redpanda** (kompatibel API Kafka v3). Keunggulannya adalah arsitektur *single binary* C++ (tanpa Java JVM dan Apache ZooKeeper), konsumsi memori rendah, latensi I/O disk sangat stabil melalui penyimpanan berbasis seastar engine, serta dukungan bawaan utilitas CLI `rpk`.
   - *Pola:* Menggunakan pola **Publish–Subscribe berbasis Log Terdistribusi** dengan konsep **Consumer Groups**. *Topic* dibagi ke dalam 3 partisi. Masing-masing aplikasi konsumen membentuk *consumer group* tersendiri, sehingga Redpanda mendistribusikan salinan pesan (*fan-out*) ke setiap grup secara independen.
2. **Semantik Pengiriman yang Berlaku (*Delivery Semantics*):** Sistem menerapkan semantik ***At-Least-Once Delivery***. Mekanisme ini diwujudkan melalui alur kerja berikut:
   - Produser mengirimkan pesan dengan tingkat pengakuan (*acknowledgement*) `acks=all` (atau `acks=1`), memastikan pesan telah tercatat pada *commit log* broker sebelum dianggap sukses.
   - Konsumen mematikan *auto-commit* (`CommitInterval: 0`). Konsumen memanggil `reader.FetchMessage(ctx)`, mendekode isi pesan JSON, menjalankan pemrosesan bisnis dan deduplikasi, dan hanya memanggil `reader.CommitMessages(ctx, msg)` apabila seluruh tahapan tersebut tidak menghasilkan kesalahan.
   - Apabila terjadi gangguan jaringan atau proses konsumen mati secara tiba-tiba di tengah eksekusi, *offset* belum berpindah pada broker. Saat proses hidup kembali, broker akan mengirimkan ulang (*redeliver*) pesan tersebut. Dengan demikian, tidak ada pesan yang hilang (*no message loss*), meskipun ada konsekuensi potensi pemrosesan ganda.
3. **Konsekuensi bagi Consumer dan Perlunya Sifat Idempoten:**

- **Mengapa Consumer Perlu Idempoten?** Dalam semantik *at-least-once*, pengiriman ganda (*duplicate delivery*) adalah keniscayaan akibat *network timeout*, kegagalan commit offset, atau rebalancing antar partisi. Jika konsumen tidak idempoten:
  - Pada layanan *Dashboard*: Metrik jumlah bencana akan terhitung ganda (*double count*), menghasilkan statistik tingkat bahaya yang tidak akurat bagi pengambil kebijakan.
  - Pada layanan *Notifier*: Petugas lapangan dan masyarakat akan menerima notifikasi darurat yang sama secara berulang-ulang (*alert fatigue* atau kepanikan publik).
- **Bagaimana Kelompok Menyikapinya?** Kelompok menerapkan strategi **Idempotent Processing with Event Identity**:
  - Setiap entitas kejadian bencana memiliki identitas global unik yang deterministik (`hazard_id`).
  - Pada level prototipe saat ini, konsumen menyimpan set `processed[hazard_id]` yang dilindungi kunci sinkronisasi (`sync.Mutex`). Sebelum memproses atau meneruskan notifikasi, konsumen memeriksa keberadaan ID tersebut. Jika ID telah tercatat, pesan ditandai sebagai `Duplicate`, diabaikan dari penambahan metrik atau pengiriman peringatan, dan *offset*-nya langsung dikomit ke broker.
  - Untuk lingkungan produksi skala penuh dengan replika ganda (*multi-replica consumers*), disikapi dengan memanfaatkan penyimpanan terdistribusi berkecepatan tinggi dengan operasi atomik (misalnya Redis dengan perintah `SETNX hazard_id "processed" EX 86400`) atau tabel basis data relasional dengan indeks unik (`INSERT ... ON CONFLICT DO NOTHING`).

Bab 4

Petunjuk Menjalankan Sistem

Bab ini menyajikan panduan ringkas operasional untuk mengonfigurasi, menjalankan, dan memverifikasi seluruh komponen *Disaster Data Aggregator Platform* pada lingkungan pengujian evaluator. Panduan lengkap dan komprehensif didokumentasikan pada berkas `README.md` di direktori utama repositori.

4.1 Prasyarat dan Konfigurasi Lingkungan

Sebelum menjalankan sistem, pastikan perangkat kerja penguji telah memenuhi kebutuhan piranti lunak berikut:

1. **Docker Engine & Docker Compose:** Docker Engine minimal versi 24.0+ dan Docker Compose Plugin minimal versi v2.20+.
2. **Go Toolchain (Opsional untuk kompilasi lokal):** Go v1.22+ (seluruh layanan telah memiliki kontainerisasi terisolasi sehingga instalasi Go lokal bersifat opsional).
3. **Utilitas Terminal:** `make`, `curl`, `jq`, dan `bash` untuk mengeksekusi skrip otomatisasi.
4. **Alat Pengujian Beban (Opsional):** `k6` (jika tidak terpasang di *host*, target pengujian `make load-test` akan secara otomatis mengunduh dan menjalankan citra kontainer `grafana/k6`).

Langkah Konfigurasi Lingkungan (*Environment Setup*):

```
# Salin template variabel lingkungan dari .env.example
cp .env.example .env

# (Opsional) Sesuaikan nilai rahasia atau port jika diperlukan
```

Listing 4.1: Inisialisasi Variabel Lingkungan

4.2 Langkah Menjalankan Sistem

Seluruh orkestrasi kontainer (*mock services*, basis data, broker Redpanda, layanan inti BNPB, dan konsumen hilir) dikelola melalui `Makefile` dan `docker-compose.yml`:

1. **Menjalankan Seluruh Platform:** Eksekusi perintah berikut untuk membangun citra kontainer dan mengaktifkan 11 kontainer layanan di latar belakang:

```
make up
# Ekuivalen dengan: docker compose up -d --build

```

   Tunggu sekitar 5–10 detik hingga kontainer `redpanda-init` selesai menginisialisasi partisi dan seluruh layanan berstatus *healthy*.

2. **Verifikasi Status Kesehatan Layanan (*Health Checks*):** Pastikan seluruh titik akhir publik dan internal merespons dengan status `"UP"`:

```
curl -s http://localhost:8081/health | jq .   # mock-bmkg
curl -s http://localhost:8082/health | jq .   # mock-pvmbg
curl -s http://localhost:8084/health | jq .   # auth service
curl -s http://localhost:8080/health | jq .   # client-api gateway
docker compose exec client-api curl -s http://aggregator:8083/health |
    jq . # aggregator

```

3. **Melihat Log Layanan Secara Kolektif:**

```
make logs

```

4. **Menghentikan Sistem dan Membersihkan Lingkungan:**

```
make down
# Ekuivalen dengan: docker compose down -v

```

4.3 Langkah Verifikasi dan Demonstrasi Solusi (Problem 1–5)

Repositori menyediakan target otomatisasi `make` terintegrasi untuk mereplikasi dan memvalidasi kriteria evaluasi setiap permasalahan:

Tabel 4.1: Matriks Verifikasi dan Target Eksekusi Demonstrasi Permasalahan

Problem	**Target Make**	**Skrip Pengujian**	**Kriteria Verifikasi**
Problem 1	`make demo-p1`	`scripts/test_schema_change.sh`	Evolusi skema aditif (`confidence_level`) diserap dinamis tanpa migrasi database atau *redeploy*.
Problem 2	`make demo-p2` `make load-test`	`scripts/test_outage.sh` `scripts/load_test.js`	Simulasi pemadaman PVMBG; respons terdegradasi (`degraded: true`); BMKG tetap normal. Uji beban 50 koneksi konstan selama 60 detik (k6) dengan tingkat keberhasilan 100%.
Problem 3	`make demo-p3`	`scripts/test_cross_credential.sh` `scripts/test_auth_scopes.sh`	Isolasi *trust domain* upstream (penolakan kredensial silang); penolakan akses `403` bagi Media pada *raw fields*; rotasi token JWT (TTL 60 detik).
Problem 4	`make demo-p4`	`scripts/test_storage_restart.sh`	*Independent container rebuild* dan *restart*; isolasi kegagalan tanpa efek kaskade ke basis data.
Problem 5	`make demo-p5`	`scripts/test_consumer_recovery.sh`	Distribusi *fan-out*; akumulasi *lag* saat *downtime*; pemulihan *offset*; penambahan konsumen ke-3 secara dinamis.

Setiap berkas log lengkap hasil demonstrasi pengujian otomatis ini diarsipkan pada repositori di direktori `docs/evidence/p1/` hingga `docs/evidence/p5/`.

Daftar Pustaka

[1] Dick Hardt. The OAuth 2.0 Authorization Framework. RFC 6749, Internet Engineering Task Force, October 2012. URL https://www.rfc-editor.org/rfc/rfc6749.

[2] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, May 2015. URL https://www.rfc-editor.org/rfc/rfc7519.

[3] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Sebastopol, CA, 2017.

[4] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA, 2nd edition, 2021.

[5] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Raleigh, NC, 2nd edition, 2018.

[6] PostgreSQL Global Development Group. Postgresql 16 documentation: Json types and functions. https://www.postgresql.org/docs/16/datatype-json.html, 2024.

[7] Redpanda Data Inc. Redpanda architecture and kafka api compatibility documentation. https://docs.redpanda.com/, 2024.

[8] Chris Richardson. *Microservices Patterns: With examples in Java*. Manning Publications, Shelter Island, NY, 2018.

Lampiran A
Pernyataan Penggunaan Kecerdasan Buatan (AI)

Sesuai dengan ketentuan penulisan pada dokumen spesifikasi tugas besar IF4031 Arsitektur Aplikasi Terdistribusi (Milestone 1), tim pengembang mendeklarasikan secara terbuka dan jujur penggunaan model bahasa besar (*Large Language Model* / AI) selama proses perancangan, implementasi, dan penyusunan laporan.

Deklarasi Penggunaan

Tim pengembang memanfaatkan alat bantu AI (*Antigravity CLI*, *Anthropic Claude*, dan *OpenAI ChatGPT*) untuk keperluan:

1. **Eksplorasi Sintaksis dan *Boilerplate*:** Pembuatan kerangka awal sintaks Go (pengurai JSON dan pustaka `kafka-go`) serta template skrip pengujian Bash dan k6.
2. ***Proofreading* dan Penyempurnaan Tata Bahasa:** Pemeriksaan konsistensi istilah teknis, keterbacaan kalimat, dan format penataan dokumen LaTeX.
3. **Perumusan Alternatif Arsitektural:** Diskusi mengenai perbandingan kelebihan dan kelemahan pola *Transactional Outbox* serta pertimbangan trade-off antara PostgreSQL JSONB dan MongoDB.

Pernyataan Tanggung Jawab dan Pemahaman

Seluruh anggota kelompok menyatakan dengan sesungguhnya bahwa:

1. Seluruh keputusan arsitektur, pemilihan teknologi, pembagian batas layanan (*bounded context*), dan logika bisnis inti dirumuskan, didiskusikan, dan diputuskan secara mandiri oleh tim pengembang.
2. Seluruh kode yang dihasilkan atau disempurnakan oleh alat bantu AI telah ditelaah, dipahami sepenuhnya, serta diverifikasi fungsionalitasnya oleh anggota kelompok penanggung jawab komponen.
3. Seluruh isi laporan ini merepresentasikan pekerjaan orisinal kelompok. Tim pengembang bertanggung jawab penuh atas kebenaran, integritas akademik, dan keandalan sistem yang diajukan.

Bandung, 8 Oktober 2026

Aldoy Fauzan Avanza	**Darrel Adinarya Sunanda**	**Jennifer Khang**
NIM: 18223113	NIM: 13523061	NIM: 13524110